

TDE – SISTEMAS DISTRIBUÍDOS

Sistema Distribuído de Consulta e Gestão de Filas do SAC Bahia

Opção 2 – REST (OpenAPI/Swagger)

Disciplina: Sistemas Distribuídos

Instituição: Centro Universitário Nobre – UNIFAN

TDE – Sistemas Distribuídos (2026.2)

Professor: Rafael Levi Batista Costa

Documento de Planejamento, Especificação e Arquitetura do Projeto

Estevão Cunha • Guilherme Fernandes • João Pedro Vinhas • Matheus da Silva • Ulysses Silva

1. Identificação do Projeto

Item	Definição
Nome	Sistema Distribuído de Consulta e Gestão de Filas do SAC Bahia
Modalidade	TDE – Sistemas Distribuídos
Opção	Opção 2 – REST (OpenAPI/Swagger)
Servidor	Node.js + Express
Cliente	Python + requests
Persistência	SQLite no ambiente do TDE
Contrato	OpenAPI 3.x
Comunicação	HTTP/REST com JSON
Adicional	Histórico de atualizações das filas

2. Contextualização e Ideia do Projeto

O projeto propõe uma solução distribuída para consulta do estado das filas de atendimento de postos do SAC Bahia. O sistema simula um cenário no qual um cliente desenvolvido em Python precisa consumir um serviço desenvolvido em Node.js, utilizando HTTP/REST e JSON como mecanismo de comunicação.

O sistema será representado por postos fictícios ou representativos para fins acadêmicos. Não haverá integração com os sistemas reais do SAC Bahia. A finalidade é demonstrar, de forma prática, os conceitos de sistemas distribuídos, contratos de interface, comunicação entre processos heterogêneos, tratamento de falhas e preparação arquitetural para escalabilidade.

2.1 Problema

Usuários de serviços presenciais precisam de informações sobre a situação das filas antes de se deslocarem até um posto. Em um cenário computacional distribuído, essas informações podem ser disponibilizadas por um serviço centralizado e consumidas por diferentes aplicações.

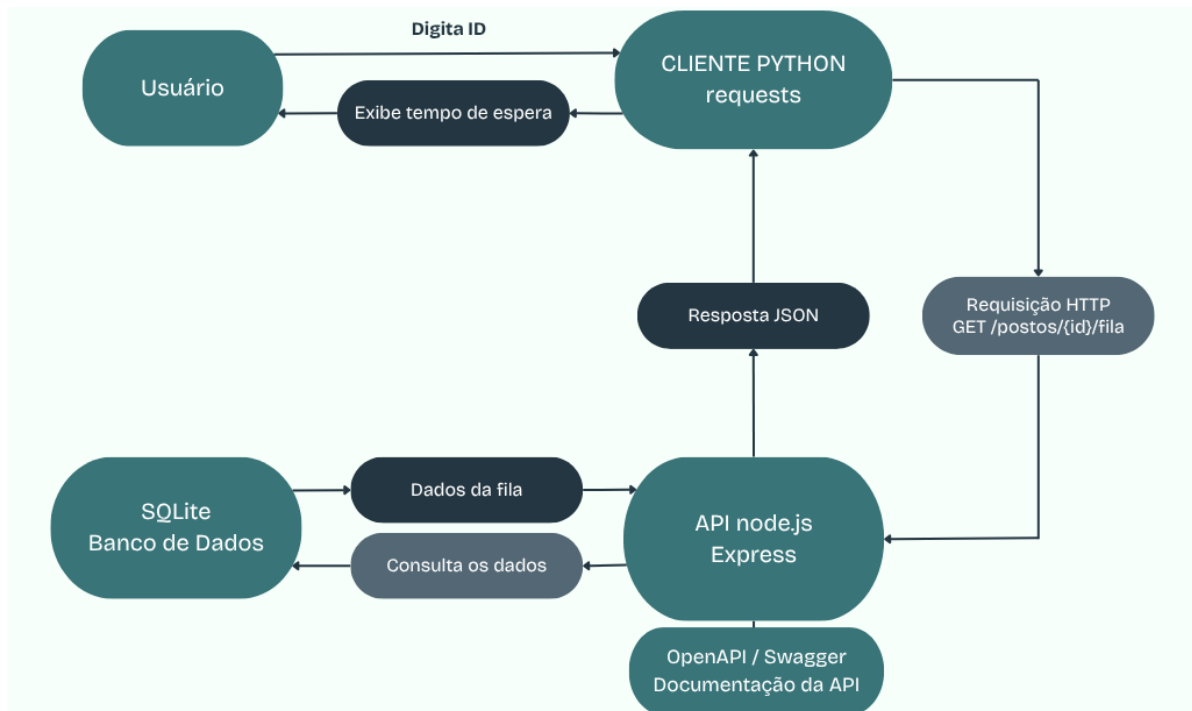
O desafio acadêmico consiste em permitir que uma aplicação cliente escrita em Python consuma uma API escrita em Node.js, sem que o cliente conheça detalhes internos do banco ou da implementação do servidor.

2.2 Justificativa

A escolha do problema permite trabalhar diretamente com os requisitos da Opção 2 do TDE: API REST em Node.js/Express, cliente Python com requests, dados JSON e rotas documentadas por OpenAPI/Swagger. O domínio também permite demonstrar atualização de estado, tratamento de erros, persistência, histórico e uma evolução arquitetural coerente para ambientes com maior volume de acessos.

3. System Design

A solução será dividida em componentes independentes. O cliente Python não acessará o banco de dados diretamente. Toda informação será obtida através da API REST, deixando explícita a comunicação entre processos e permitindo que cliente e servidor possam, conceitualmente, executar em máquinas diferentes.



4. Objetivos

4.1 Objetivo Geral

Implementar uma solução distribuída em que um cliente Python consulte e consuma, por HTTP/REST, uma API Node.js/Express responsável por fornecer informações sobre postos, filas de atendimento e tempo estimado de espera, utilizando OpenAPI como contrato da interface e JSON como formato de troca de dados.

4.2 Objetivos Específicos

- Implementar uma API RESTful utilizando Node.js e Express.
- Criar um cliente Python utilizando a biblioteca requests.
- Estabelecer comunicação entre processos escritos em linguagens diferentes por HTTP.
- Persistir os dados dos postos e filas em SQLite.
- Diferenciar a quantidade de pessoas em fila normal e prioritária.
- Calcular uma estimativa de tempo de espera conforme regra definida no projeto.
- Permitir consulta dos postos disponíveis.
- Permitir consulta de um posto específico.
- Permitir consulta da fila de um posto.
- Permitir atualização controlada dos dados da fila para demonstração.
- Registrar histórico das atualizações de fila como funcionalidade adicional.
- Documentar o contrato da API com OpenAPI/Swagger.
- Implementar validação de entradas e respostas JSON padronizadas.
- Implementar rate limiting e tratamento centralizado de erros.
- Demonstrar que cliente e servidor podem operar em processos e máquinas distintas.

- Preparar a arquitetura para futura migração de SQLite para PostgreSQL e escalonamento horizontal.

5. Escopo

5.1 Dentro do escopo

- Cadastro inicial de postos fictícios/representativos.
- Listagem de postos disponíveis.
- Consulta de posto por ID.
- Consulta da fila de um posto.
- Separação entre fila normal e prioritária.
- Registro de guichês ativos e tempo médio de atendimento.
- Cálculo de tempo estimado de espera.
- Atualização dos dados da fila para demonstração.
- Registro histórico das alterações de fila.
- Cliente Python em terminal.
- API REST Node.js/Express.
- Banco SQLite.
- Contrato OpenAPI.
- Swagger UI para documentação e testes.
- Tratamento de erros HTTP.
- Rate limiting.
- Testes automatizados básicos.
- README com instruções de instalação e execução.
- Demonstração da comunicação em rede.

5.2 Fora do escopo

- Integração com sistemas reais do SAC Bahia.
- Aplicativo mobile.
- Frontend web complexo.
- Geolocalização.
- Pagamentos.
- Agendamento real.
- Autenticação de usuários finais.
- Infraestrutura cloud obrigatória para a entrega.
- Kubernetes e orquestração de contêineres.
- Inteligência artificial.
- Integrações externas não necessárias ao TDE.

Observação: PostgreSQL, Redis, Load Balancer, HTTPS e múltiplas instâncias da API serão tratados como evolução arquitetural e escalabilidade, não como dependências obrigatórias do MVP.

6. Requisitos Funcionais

ID	Requisito
RF01	Disponibilizar a lista de postos cadastrados.

RF02	Permitir consultar um posto pelo seu ID.
RF03	Permitir consultar a fila de um posto.
RF04	Retornar quantidade de pessoas na fila normal.
RF05	Retornar quantidade de pessoas na fila prioritária.
RF06	Retornar quantidade de guichês ativos.
RF07	Retornar o tempo médio de atendimento utilizado na estimativa.
RF08	Retornar o tempo estimado de espera.
RF09	Permitir atualizar os dados da fila para demonstração.
RF10	Registrar uma atualização no histórico da fila.
RF11	Permitir consultar o histórico de atualizações de um posto.
RF12	Disponibilizar endpoint de verificação de saúde da API.
RF13	Permitir ao cliente Python selecionar um posto e consultar sua fila.
RF14	Informar erros de forma padronizada em JSON.

7. Requisitos Não Funcionais

ID	Requisito
RNF01	A comunicação deverá utilizar HTTP/REST.
RNF02	As respostas da API deverão utilizar JSON.
RNF03	O servidor deverá ser implementado em Node.js com Express.
RNF04	O cliente deverá ser implementado em Python utilizando requests.
RNF05	O contrato da API deverá ser documentado em OpenAPI.
RNF06	A persistência do MVP deverá utilizar SQLite.
RNF07	O cliente deverá utilizar timeout nas requisições.
RNF08	A API deverá possuir validação de entrada.
RNF09	A API deverá possuir tratamento centralizado de erros.
RNF10	A API deverá limitar requisições por cliente/IP.
RNF11	Segredos e configurações locais não deverão ser versionados.
RNF12	A aplicação deverá ser executável seguindo as instruções do README.
RNF13	A arquitetura deverá permitir futura separação entre múltiplas instâncias da API e banco compartilhado.
RNF14	A documentação deverá permitir compreender e testar os endpoints sem consultar o código-fonte.

8. Regras de Negócio

ID	Regra
RN01	Cada posto possui um identificador único.
RN02	As quantidades de pessoas não podem ser negativas.
RN03	A quantidade de guichês ativos deve ser maior que zero para cálculo automático.
RN04	O tempo médio de atendimento deve ser positivo.
RN05	A fila normal e a fila prioritária serão armazenadas separadamente.
RN06	O tempo estimado será tratado como estimativa acadêmica, não como previsão oficial do SAC.
RN07	Cada atualização válida da fila deverá gerar um registro no histórico.
RN08	Um posto inexistente deverá gerar resposta HTTP 404.
RN09	Parâmetros inválidos deverão gerar resposta HTTP 400.

RN10	Excesso de requisições deverá gerar resposta HTTP 429.
RN11	Erros internos não deverão expor stack trace ao cliente.
RN12	O endpoint de atualização poderá ser protegido por API Key simples para demonstração.

9. Modelo de Dados

O modelo foi ajustado para representar corretamente a regra de negócio de filas normais e prioritárias e para suportar o adicional de histórico.

Tabela	Campo	Tipo	Descrição
postos	id	INTEGER PK	Identificador do posto
postos	nome	TEXT	Nome do posto
postos	cidade	TEXT	Cidade
postos	endereco	TEXT	Endereço representativo
postos	ativo	BOOLEAN	Indica se o posto está ativo
postos	criado_em	DATETIME	Data de criação
filas	id	INTEGER PK	Identificador da fila
filas	posto_id	INTEGER FK	Posto associado
filas	personas_normais	INTEGER	Quantidade na fila normal
filas	personas_prioritarias	INTEGER	Quantidade prioritária
filas	guiches_ativos	INTEGER	Guichês em operação
filas	tempo_medio_atendimento	INTEGER	Tempo médio em minutos
filas	tempo_estimado	INTEGER	Estimativa em minutos
filas	atualizado_em	DATETIME	Última atualização
historico_filas	id	INTEGER PK	Identificador do registro
historico_filas	posto_id	INTEGER FK	Posto associado
historico_filas	personas_normais	INTEGER	Estado registrado
historico_filas	personas_prioritarias	INTEGER	Estado registrado
historico_filas	guiches_ativos	INTEGER	Estado registrado
historico_filas	tempo_estimado	INTEGER	Estimativa registrada
historico_filas	registrado_em	DATETIME	Momento do registro

10. Cálculo da Estimativa de Espera

Para evitar a promessa de uma previsão exata, o sistema utilizará o termo tempo estimado. No MVP, a estimativa será calculada por uma regra simples e transparente:

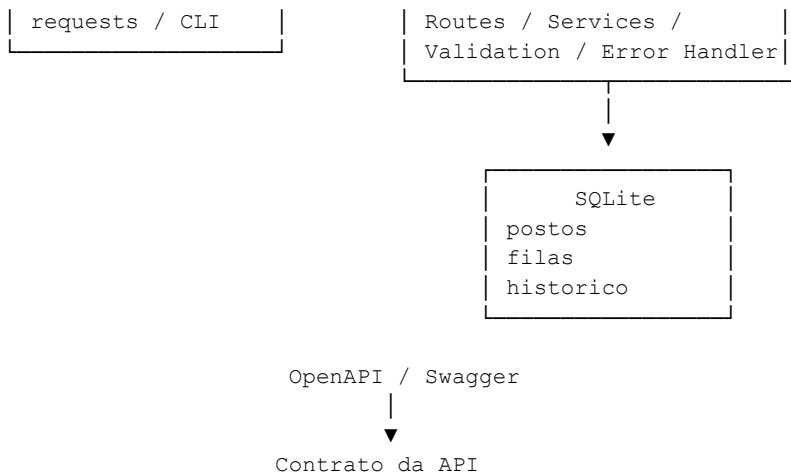
```
carga_total = pessoas_normais + pessoas_prioritarias
etapas = ceil(carga_total / guiches_ativos)
tempo_estimado = etapas * tempo_medio_atendimento
```

A fórmula é uma aproximação acadêmica destinada a demonstrar processamento de regra de negócio. Ela não representa o algoritmo oficial de cálculo utilizado pelo SAC Bahia.

11. Arquitetura da Solução

A arquitetura utiliza separação entre cliente, API e persistência. O cliente Python nunca acessa diretamente o banco. Todas as operações passam pelo contrato HTTP definido em OpenAPI.





11.1 Responsabilidades dos Componentes

Componente	Responsabilidade
Cliente Python	Receber entrada do usuário, chamar API, interpretar respostas e exibir resultados.
API Node.js/Express	Receber requisições, validar dados, aplicar regras e retornar respostas.
Services	Concentrar regras de negócio e cálculo da estimativa.
Database	Persistir postos, filas e histórico.
OpenAPI	Descrever o contrato da API.
Swagger UI	Permitir visualização e testes manuais do contrato.
Rate Limiter	Limitar volume de requisições por cliente/IP.
Error Handler	Padronizar respostas de erro em JSON.

12. API REST – Contrato

Método	Endpoint	Finalidade	Status principais
GET	/postos	Lista postos	200
GET	/postos/{id}	Consulta posto	200, 400, 404
GET	/postos/{id}/fila	Consulta fila	200, 400, 404
PUT	/postos/{id}/fila	Atualiza fila	200, 400, 401/403, 404
GET	/postos/{id}/historico	Consulta histórico	200, 400, 404
GET	/health	Verifica saúde da API	200, 503

12.1 Exemplo – Consulta de Fila

GET /postos/2/fila

```

{
  "posto": {
    "id": 2,
    "nome": "SAC Feira de Santana",
    "cidade": "Feira de Santana"
  },
  "fila": {
    "pessoas_normais": 18,
    "pessoas_prioritarias": 4,
    "guiches_ativos": 3,
    "tempo_medio_atendimento_minutos": 5,
    "tempo_estimado_minutos": 40,
    "atualizado_em": "2026-09-20T10:30:00"
  }
}

```

```
}  
}
```

12.2 Exemplo – Atualização

```
PUT /postos/2/fila  
  
{  
  "pessoas_normais": 20,  
  "pessoas_prioritarias": 5,  
  "guiches_ativos": 3,  
  "tempo_medio_atendimento_minutos": 5  
}
```

12.3 Exemplo – Erro

```
HTTP/1.1 404 Not Found  
  
{  
  "erro": {  
    "codigo": "POSTO_NAO_ENCONTRADO",  
    "mensagem": "O posto informado não existe."  
  }  
}
```

13. OpenAPI como Contrato/IDL

O arquivo docs/openapi.yaml será tratado como a fonte de verdade do contrato HTTP. Ele deverá descrever endpoints, parâmetros, corpos de requisição, schemas, respostas, códigos HTTP e exemplos.

A escolha está alinhada ao material do TDE, que apresenta OpenAPI/Swagger como a especificação utilizada para descrever contratos de APIs REST. O objetivo é permitir que o cliente Python e o servidor Node.js compartilhem uma definição comum da interface, independentemente da linguagem.

- OpenAPI define o contrato.
- Node.js implementa o servidor conforme o contrato.
- Python implementa o consumidor conforme o contrato.
- HTTP realiza o transporte.
- JSON representa os dados.
- A independência entre as linguagens demonstra a transparência de acesso.

14. Cliente Python

O cliente será executado no terminal. Ele deverá apresentar os postos disponíveis, permitir a seleção de um ID e consultar a fila correspondente. A URL base da API não ficará obrigatoriamente fixa no código, permitindo demonstrar comunicação entre máquinas diferentes.

Exemplo de execução:

```
$ python cliente.py
```

```
=====
CONSULTA SAC BAHIA
=====
```


- 1 - SAC Shopping da Bahia
- 2 - SAC Feira de Santana

Digite o ID do posto: 2

SAC Feira de Santana
Fila normal: 18
Fila prioritária: 4
Guichês ativos: 3
Tempo estimado: 40 minutos

14.1 Tratamento de Falhas no Cliente

- Timeout de conexão.
- Servidor indisponível.
- Resposta HTTP 400.
- Resposta HTTP 404.
- Resposta HTTP 429.
- Resposta HTTP 5xx.
- Resposta que não contenha JSON válido.
- Entrada de usuário que não seja um ID inteiro.

15. Segurança

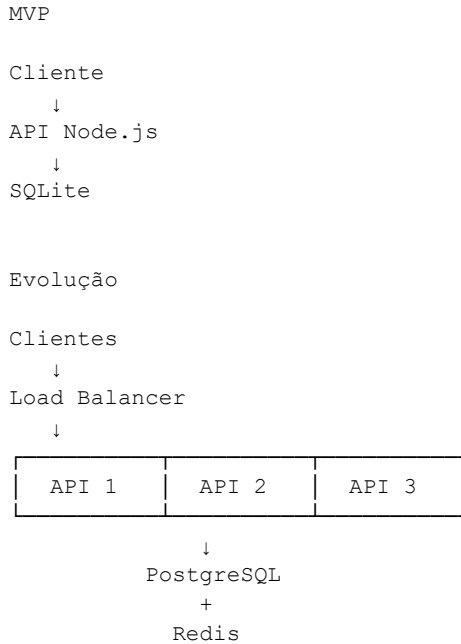
A segurança será aplicada em nível compatível com o escopo acadêmico. O objetivo é demonstrar que o serviço não depende apenas do funcionamento nominal e possui controles básicos contra entradas inválidas, abuso e exposição indevida de informações.

- Queries parametrizadas para evitar construção insegura de SQL.
- Validação de parâmetros de rota.
- Validação do corpo do PUT.
- Bloqueio de valores negativos.
- Rate limiting com express-rate-limit ou mecanismo equivalente.
- Helmet para headers de segurança HTTP.
- Variáveis de ambiente para configurações.
- .env incluído no .gitignore.
- .env.example disponibilizado para configuração.
- Tratamento global de erros.
- Nenhum stack trace será enviado ao cliente.
- API Key simples poderá proteger o endpoint de atualização.
- Uso de HTTPS será previsto na arquitetura de produção.
- npm audit será executado antes da entrega.

Limite importante: Rate limiting reduz abuso na camada de aplicação, mas não deve ser apresentado como mecanismo completo de proteção contra ataques DDoS. Proteções de borda, WAF, CDN ou infraestrutura especializada pertencem ao ambiente de produção.

16. Escalabilidade

SQLite será mantido no MVP por simplicidade e adequação ao TDE. Para um ambiente produtivo com múltiplos postos e alta concorrência, a arquitetura deverá migrar para banco compartilhado e suportar múltiplas instâncias da API.



- Migração do SQLite para PostgreSQL.
- API stateless para facilitar replicação.
- Load Balancer para distribuição das requisições.
- Redis como possibilidade para cache e estado compartilhado de rate limiting.
- Endpoint /health para verificar disponibilidade da instância.
- Cache de consultas de leitura como evolução.
- Logs e métricas para observabilidade.
- Teste de carga como evidência complementar, se houver tempo.

17. Diferencial do Projeto

O diferencial escolhido é o registro histórico da evolução das filas. A funcionalidade transforma uma consulta pontual em um histórico do comportamento do atendimento, sem exigir frontend, aplicativo ou integrações externas.

- Cada atualização válida gera um registro.
- O histórico pode ser consultado pelo endpoint GET /postos/{id}/historico.
- O histórico demonstra persistência de eventos.
- A funcionalidade cria base para futuras análises de fluxo.
- O recurso é pequeno o suficiente para permanecer dentro do escopo do TDE.

18. Transparência de Acesso

A transparência será demonstrada pela capacidade de um processo Python utilizar um serviço Node.js sem precisar conhecer sua implementação interna. O cliente conhece somente o contrato HTTP.

1. O usuário inicia o cliente Python.
2. O cliente solicita ou apresenta os postos disponíveis.
3. O usuário informa o ID do posto.
4. O Python envia GET /postos/{id}/fila.
5. A API Node.js recebe a requisição.
6. A API valida o parâmetro.
7. O serviço consulta o banco.
8. A API monta a resposta conforme OpenAPI.
9. O JSON é enviado ao Python.
10. O cliente interpreta o JSON e apresenta o resultado.

A demonstração poderá ser feita com cliente e servidor em máquinas diferentes, desde que estejam na mesma rede e que a porta da API esteja acessível. A URL da API deverá ser configurável no cliente.

19. Git, Organização e Colaboração

- Repositório oficial no GitHub.
- Branch main destinada a código estável.
- Branches feature/* e fix/* para desenvolvimento.
- Pull Requests para integração.
- Revisão por outro integrante quando possível.
- Conventional Commits.
- Issues para acompanhamento de tarefas.
- GitHub Actions para testes.
- Tag de versão v1.0.0 na entrega.
- README completo.
- Todos os integrantes devem possuir participação rastreável, conforme a organização definida pela equipe.

Exemplos:

```
feat: implementa consulta de fila
feat: adiciona historico de filas
fix: corrige validacao do id
test: adiciona testes da rota de fila
docs: atualiza openapi
refactor: separa camada de servico
```

20. Estrutura do Repositório

```
tde-sistemas-distribuidos/
|
|— servidor-node/
|   |— src/
|   |   |— routes/
|   |   |— controllers/
|   |   |— services/
```

```

├── middlewares/
│   └── database/
├── tests/
├── app.js
├── package.json
├── .env.example
├── cliente-python/
│   ├── cliente.py
│   ├── requirements.txt
│   └── .env.example
├── docs/
│   ├── openapi.yaml
│   ├── arquitetura.png
│   ├── sequencia.png
│   └── modelo-dados.png
├── .github/
│   └── workflows/
│       └── tests.yml
├── .gitignore
└── README.md

```

21. Testes e Critérios de Aceitação

Teste	Resultado esperado
Servidor inicia	Processo Node.js inicia sem erros.
GET /postos	HTTP 200 e JSON com postos.
GET /postos/{id}	HTTP 200 para posto existente.
ID inválido	HTTP 400 e JSON padronizado.
Posto inexistente	HTTP 404 e JSON padronizado.
Consulta de fila	HTTP 200 com dados da fila.
Atualização	HTTP 200, dados atualizados e histórico criado.
Histórico	HTTP 200 com registros de atualizações.
Rate limit	HTTP 429 quando limite for excedido.
Erro interno	HTTP 500 sem stack trace.
Cliente Python	Consulta e exibição do resultado.
Servidor indisponível	Cliente informa falha de conexão.
Timeout	Cliente encerra espera conforme limite.
Máquinas distintas	Cliente consegue acessar API pela rede.
Swagger	Endpoints descritos e testáveis.
CI	Testes executados automaticamente em push/PR.

22. Plano de Implementação

Etapas	Atividade	Resultado
1	Preparar repositório e ambientes	Node/Python/venv configurados
2	Modelar banco	Tabelas e dados iniciais
3	Implementar API base	Servidor e rotas principais
4	Implementar regras de negócio	Validação e estimativa
5	Implementar histórico	Adicional funcional
6	Implementar segurança	Rate limit, validação, erros
7	Criar OpenAPI/Swagger	Contrato completo
8	Implementar cliente Python	Consumidor funcional
9	Testes	Casos positivos e negativos

10	CI/GitHub	Workflow automatizado
11	System Design	Diagramas e documentação
12	Integração em rede	Node e Python em processos/máquinas distintas
13	Revisão	Checklist de requisitos e rubrica
14	Entrega	README, código e apresentação

23. Checklist da Rubrica de Avaliação

Critério	Peso	Evidências previstas
1. Montagem e Gestão do Grupo	0,5	Divisão de responsabilidades, cronograma, Issues, branches e participação rastreável.
2. Planejamento de Segurança Cloud	0,5	Validação, queries parametrizadas, rate limit, Helmet, .env, tratamento de erros, arquitetura TLS.
3. Domínio do Negócio	0,5	Postos, filas normal/prioritária, guichês, ETA, regras de negócio e histórico.
4. Planejamento de Escalonamento	0,5	PostgreSQL, API stateless, Load Balancer, Redis/cache, /health e observabilidade.
5. Escolha do Diferencial	0,5	Histórico da evolução das filas.
6. Organização do Repositório (Git)	0,5	GitHub, branches, PRs, Conventional Commits, CI, testes, README.
7. Arquitetura e System Design	0,5	Diagrama de componentes, sequência, modelo de dados e arquitetura de evolução.

23.1 Rubrica Oficial do TDE

Critério oficial	Peso	Como será atendido
Contrato/IDL	1,0	OpenAPI como contrato, com endpoints, schemas, parâmetros, respostas e exemplos.
Backend	1,0	Node.js/Express funcionando, persistência SQLite, regras, segurança e respostas JSON.
Frontend/Cliente	1,0	Cliente Python + requests, tratamento de respostas e falhas.
Transparência	1,0	Comunicação real Node.js ↔ Python por HTTP/JSON, inclusive possibilidade de máquinas distintas.

24. Checklist Final de Entrega

Projeto e domínio

- ☐ Escopo definido
- ☐ Postos representativos definidos
- ☐ Fila normal e prioritária implementadas
- ☐ Estimativa documentada
- ☐ Histórico implementado

Backend

- ☐ Node.js/Express funcionando
- ☐ Todas as rotas funcionando
- ☐ JSON padronizado
- ☐ Códigos HTTP corretos
- ☐ Validação implementada
- ☐ Tratamento global de erros
- ☐ Rate limit funcionando
- ☐ Health check funcionando

Banco

- ☐ SQLite funcionando
- ☐ Foreign keys
- ☐ Seed inicial
- ☐ Atualização de fila
- ☐ Histórico

Cliente

- ☐ Python + requests
- ☐ venv documentado
- ☐ Timeout
- ☐ Tratamento de conexão
- ☐ Tratamento de 400/404/429/5xx
- ☐ URL configurável

Contrato

- ☐ openapi.yaml completo
- ☐ Schemas
- ☐ Request bodies
- ☐ Responses
- ☐ Swagger testado

Segurança

- ☐ Queries parametrizadas
- ☐ Validação
- ☐ .env
- ☐ .env.example
- ☐ .gitignore
- ☐ Helmet
- ☐ Rate limiting
- ☐ Sem stack trace

Git

- ☐ GitHub
- ☐ Branches
- ☐ PRs
- ☐ Conventional Commits
- ☐ CI
- ☐ README
- ☐ Release v1.0.0

System Design

- ☐ Diagrama de componentes
- ☐ Diagrama de sequência
- ☐ Modelo de dados
- ☐ Arquitetura de escalabilidade

Demonstração

- ☐ Servidor iniciado
- ☐ Swagger demonstrado
- ☐ Cliente Python executado
- ☐ Comunicação Node ↔ Python comprovada
- ☐ Teste de erro 404
- ☐ Teste de atualização
- ☐ Consulta do histórico
- ☐ Demonstração em rede, se disponível

25. Viabilidade e Aplicabilidade

Dimensão	Avaliação
Técnica	Alta para o escopo acadêmico, pois utiliza tecnologias simples e compatíveis com o enunciado.
Acadêmica	Alta aderência à Opção 2 e aos critérios de contrato, backend, cliente e transparência.
Implementação	Viável sem frontend, cloud obrigatória ou infraestrutura complexa.
Demonstração	Alta: API, Swagger, cliente Python, erros e histórico são facilmente demonstráveis.
Evolução	Permite PostgreSQL, Redis, balanceamento e múltiplas instâncias sem alterar o conceito central.

26. Possíveis Evoluções Futuras

- Migração para PostgreSQL.
- Redis para cache.
- Balanceador de carga.
- Deploy em cloud.
- HTTPS/TLS em produção.

- Dashboard web.
- Aplicativo móvel.
- Gráficos do histórico.
- Integração com sistemas oficiais, caso exista autorização e API.
- Monitoramento com métricas e alertas.
- Estimativa baseada em dados históricos reais.

27. Resumo para Apresentação

A equipe desenvolveu um sistema distribuído para consulta e gestão simulada de filas do SAC Bahia. A solução possui um servidor Node.js/Express e um cliente Python que se comunicam por HTTP/REST, utilizando JSON e um contrato OpenAPI. O servidor mantém os dados em SQLite e disponibiliza operações de consulta e atualização das filas. O sistema diferencia filas normais e prioritárias, calcula uma estimativa de espera e registra o histórico das alterações. O projeto também implementa validação, tratamento padronizado de erros, rate limiting e documentação da API. A arquitetura foi planejada para evoluir de SQLite para PostgreSQL e permitir múltiplas instâncias da API em um ambiente de maior escala.

28. Referência ao Material do TDE

Este documento foi estruturado a partir do enunciado oficial e do planejamento previamente elaborado para o projeto. O enunciado define a Opção 2 como uma API REST em Node.js/Express consumida por cliente Python com requests, com respostas em JSON e rotas REST. A rubrica oficial avalia Contrato/IDL, Backend, Cliente e Transparência. O planejamento inicial foi ampliado e corrigido para alinhar domínio, segurança, escalabilidade, Git e System Design às métricas adicionais de avaliação fornecidas pela equipe.